

TypeScript

Introducción y conceptos fundamentales

Tipos · Variables · Constantes

Objetos · Arreglos · Interfaces · Clases

Funciones · Módulos · Genéricos · Decoradores

¿Qué es TypeScript?

- Lenguaje desarrollado por Microsoft basado en JavaScript.
- Superconjunto de JavaScript (80%)
- El código TypeScript se compila/transpila a JavaScript para ejecutarse.
- Añade un sistema de tipado estático, clases,
- Mejora la legibilidad
- Ayuda a detectar errores durante el desarrollo (Detección errores en el editor)
- Facilita el desarrollo de aplicaciones grandes y mantenibles.

Tipos básicos

string → texto

number → números

boolean → verdadero/falso

date → fecha

any → cualquier tipo (evitar)

unknown → tipo desconocido

También existen null, undefined, void y never

```
let nombre: string = "Ana";  
let edad: number = 20;  
let activo: boolean = true;
```

Constantes y Variables

CONSTANTES

- Se declaran con const
- Útiles para valores que no deben ser reasignados
- Mas ligeras

```
const PI: number = 3.1416;  
const IVA: number = 0.21;  
const PAIS: string = "España";
```

VARIABLES

- let: el valor puede cambiar
- const: no permite reasignar la variable
- Operado “|” → Permite definir diferentes tipos aceptados

```
let edad: number = 18;  
edad = 19;  
  
const nombre: string = "Laura";
```

```
let nivelCarga: number | string ;  
nivelCarga = 85;  
nivelCarga = 'FULL';  
let nivelCarga: number | 'FULL' ;
```

Objetos

Los objetos agrupan propiedades relacionadas.

Permiten representar personas, productos, coches, usuarios, etc.

```
const persona: {  
  nombre: string;  
  edad: number;  
} = {  
  nombre: "Carlos",  
  edad: 25  
};
```



TypeScript

Arrays

Almacenan varios elementos (colecciones de elementos)

Se puede indicar el tipo de sus elementos (recomendado)

Admite tipos simples, interfaces, clases, etc.

```
let numeros: number[] = [1, 2, 3, 4];  
let edades: Array<number> = [18, 20, 25];  
  
let nombres: string[] = ["Ana", "Luis", "Carlos"];  
  
let varios = ['azul', 'rojo', true, fff];  
→ Let varios = string[] = ['azul', 'rojo']
```

Arrays: Funciones principales

```
const numeros: number[] = [1, 2, 3, 4, 5];

// map
const dobles: number[] = numeros.map(n => n * 2);

// filter
const mayores: number[] = numeros.filter(n => n > 2);

// find
const resultado: number | undefined = numeros.find(n => n > 3);

// reduce
const suma: number = numeros.reduce((total, n) => total + n, 0);

console.log(dobles);    // [2, 4, 6, 8, 10]
console.log(mayores);   // [3, 4, 5]
console.log(resultado); // 4
console.log(suma);      // 15
```

Interfaces

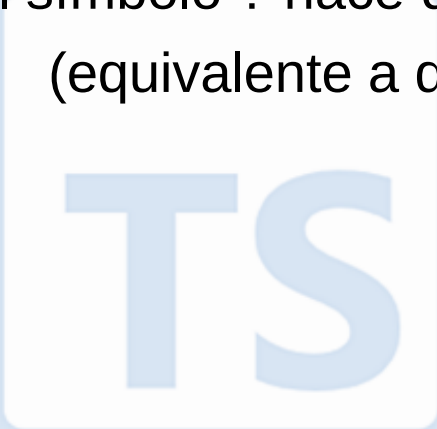
Definen la estructura que debe tener un objeto

Permiten reutilizar una misma estructura

No se transpila a JavaScript (1000 interfaces = 0 líneas código JavaScripts)

El símbolo ? hace que una propiedad sea opcional

- (equivalente a definir como tipo | undefined)



```
interface Usuario {  
  nombre: string;  
  edad: number;  
  email: string;  
  ciudad?: string  
}  
  
const usuario: Usuario = {  
  nombre: "Laura",  
  edad: 22,  
  email: "laura@email.com"  
};
```

Relación Objetos , Arrays , Interfaces

Los objetos agrupan propiedades relacionadas.

Los arreglos almacenan colecciones de elementos.

Las interfaces describen la estructura de los objetos.

Permiten reutilizar y comprobar estructuras.



TypeScript

Clases

Permiten crear objetos con propiedades y métodos

El constructor inicializa el objeto

The logo consists of the letters 'TS' in a light blue, sans-serif font, centered within a white square with rounded corners.

```
class Coche {  
  marca: string;  
  modelo: string;  
  
  constructor(marca: string, modelo: string) {  
    this.marca = marca;  
    this.modelo = modelo;  
  }  
}
```

Interfaces + Clases

Una clase puede implementar una interfaz
implements obliga a cumplir su estructura

```
interface Persona {  
  nombre: string;  
  saludar(): void;  
}  
  
class Usuario implements Persona {  
  nombre: string;  
  saludar(): void {  
    console.log(`Hola, soy ${this.nombre}`);  
  }  
}
```

4. Funciones y sus argumentos

Las funciones pueden indicar tipos para argumentos y retorno.

- (void, string, int,...)

Los argumentos pueden ser obligatorios u opcionales (?) y asignar un valor por defecto (=valor)

- function multiplicar (firstNumber:number, seconNumber?: number, base: number =2)
- Recomendado siempre definir tipo y evitar any
- Configuracion strict="true"

Posibilidad de usar funciones de flecha

```
function sumar(a: number, b: number): number
{
    return a + b;
}
let result: number = sumar(1,2);
console.log({result});
```

```
const sumarStr = (a: number, b: number): string => {
    return `${a + b}`;
}
let result2: string = sumarstr(1,2);
console.log({result2});
```

5. Desestructuración de arreglos y objetos

Permite extraer valores directamente de arreglos u objetos.

Hace el código más compacto y legible.

Es muy habitual al trabajar con datos y funciones.



TypeScript

```
const persona = { nombre: "Ana", edad: 20 };  
const { nombre, edad } = persona;  
  
const numeros = [10, 20];  
const [primero, segundo] = numeros;
```

6. Importaciones y exportaciones

Permiten dividir un programa en módulos.

export hace disponible una función, clase o variable.

import permite utilizar elementos de otro módulo.

The logo consists of the letters 'T' and 'S' in a light blue, sans-serif font, positioned on a white square background.

```
// usuario.ts
export interface Usuario { nombre: string; }

// app.ts
import { Usuario } from "../usuario";
```

7. Clases y constructores

Las clases permiten crear objetos con propiedades y métodos.

El constructor inicializa las propiedades del objeto.

implements permite cumplir una interfaz.



```
class Coche {  
  constructor(  
    public marca: string,  
    public modelo: string  
  ) {}  
  
  mostrar(): void {  
    console.log(this.marca);  
  }  
}
```

8. Tipos genéricos

Los genéricos permiten crear código reutilizable y tipado.

El tipo se determina al utilizar la función, clase o interfaz.

Se representan normalmente con <T>.



```
function identidad<T>(valor: T): T {  
    return valor;  
}  
  
const numero = identidad<number>(10);  
const texto = identidad<string>("Hola");
```

cript

9. Decoradores

Son funciones especiales asociadas a clases y otros elementos.

Permiten añadir comportamiento o metadatos.

Su disponibilidad y sintaxis dependen de la configuración y versión de TypeScript.



```
@decorador
class Usuario {
  // ...
}

function decorador(target: Function) {
  console.log("Clase:", target.name);
}
```

10. Encadenamiento opcional

El operador `?.` permite acceder a propiedades sin provocar un error si un valor es `null` o `undefined`. Puede utilizarse con propiedades, métodos y elementos.



```
const usuario = {  
  perfil: {  
    nombre: "Ana"  
  }  
};  
  
console.log(usuario.perfil?.nombre);  
console.log(usuario.direccion?.calle);
```

Resumen

Tipos básicos → seguridad y comprobación (string, number, boolean, ...)

Constantes → `const` | Variable → `let`

Objetos, arreglos e interfaces → estructuras de datos

Funciones → comportamiento y argumentos

Desestructuración → acceso sencillo a datos

Import/export → organización en módulos

Clases → programación orientada a objetos

Genéricos → reutilización con tipos

Decoradores → comportamiento/metadatos

? . → acceso seguro a propiedades

TypeScript = JavaScript + tipos + código más estructurado y mantenible